Московский авиационный институт
(национальный исследовательский университет)

Кафедра 806 «Вычислительная математика и программирование»

01.03.02 «Прикладная математика и информатика»

Объектно-ориентированное программирование (семестр 1)

ЛАБОРАТОРНАЯ РАБОТА 1

ОСНОВЫ C++ — ОДНА ПРОГРАММА НА ДИНАМИЧЕСКИХ СТРУКТУРАХ ДАННЫХ

Вариант 2

Выполнил: Яшнов В.А.

Группа: М8О-203БВ-25

Преподаватель: Дзюба Д.В.

Москва, 2026

Содержание

1 Условие задачи **2**
1.1 Цель работы . . . 2
1.2 Семейство A: линейный динамический массив . . . 2
1.3 Вариант 2 . . . 2
1.4 Обязательства программы . . . 2
1.5 Запрещённые конструкции . . . 2
1.6 Часть 2: сборка и тестирование . . . 3

2 Описание программы **3**
2.1 Архитектура проекта . . . 3
2.2 Состав функций . . . 3
2.3 Алгоритм варианта . . . 4
2.4 Состав меню . . . 4
2.5 Ввод, ошибки и владение памятью . . . 4
2.6 Сборка и запуск . . . 5
2.7 Исходный код . . . 5

3 Тестирование **19**
3.1 Покрытые сценарии . . . 20
3.2 Контрольный пример . . . 20
3.3 Результат . . . 20

4 Выводы **20**

5 Список литературы **21**

1 Условие задачи

1.1 Цель работы

Написать одну законченную консольную программу — интерактивный справочник, работающий с динамической структурой данных заданного варианта и выполняющий над ней набор операций, выбираемых из текстового меню. Вся логика оформляется как библиотека чистых функций: функции не вводят и не выводят данные сами, а принимают их параметрами, изменяют переданные структуры (в том числе по ссылке через out-параметры) и возвращают результат. Работа состоит из двух частей:

1. Единая программа варианта: динамическая структура данных, базовые операции семейства, алгоритм варианта и поиск с возвратом результата и найденного значения по ссылке.

2. Проект CMake с Google Test, который проверяет чистые функции программы.

1.2 Семейство A: линейный динамический массив

Вариант закреплён за одним из трёх семейств структур данных. Варианты 1–8 относятся к семейству A — линейному динамическому массиву. Базовые операции реализуются в `array_ops.h` / `array_ops.cpp`: создание, освобождение, изменение размера, вставка элемента, удаление элемента, печать.

1.3 Вариант 2

Сортировка выбором по возрастанию; затем `void array_reverse(int* arr, size_t size)` — развернуть массив на месте двумя указателями. Программа выводит массив после сортировки и после разворота.

Контрольный пример из задания:

$$\{3,\ 1,\ 4,\ 1,\ 5\} \longrightarrow \{1,\ 1,\ 3,\ 4,\ 5\} \longrightarrow \{5,\ 4,\ 3,\ 1,\ 1\}.$$

1.4 Обязательства программы

- Никаких утечек памяти: каждая созданная через `new[]` структура освобождается через `delete[]`, после чего указатель обнуляется: `delete[] arr;arr = nullptr;`.

- Корректный ввод: при вводе чисел и меню проверяется успешность `std::cin` с последующим `std::cin.clear()`, программа не падает при некорректном вводе.

- Устойчивость к пустой структуре: функции поиска и алгоритмы корректно работают с массивом длины 0 и возвращают `false` либо безопасное значение.

- Каждая операция выводит понятный результат: сам массив, длину, найденные значения.

1.5 Запрещённые конструкции

Запрещены:

- классы, конструкторы, семантика копирования и перемещения;

- `std::string`, `std::vector`, `std::array`;
- умные указатели.

Работа с памятью — только вручную (`new[]` / `delete[]`), строки — только C-строки.

1.6 Часть 2: сборка и тестирование

Требуется проект CMake и не менее 10 тестов Google Test, проверяющих создание и удаление массива, вставку в середину, удаление и алгоритм варианта на контрольном примере. Общие параметры работы: язык C++20, система сборки CMake 3.16+, тестирование — Google Test.

2 Описание программы

2.1 Архитектура проекта

Логика программы разделена на два модуля, что обеспечивает требование о чистых функциях и позволяет тестировать их без ввода-вывода:

- `array_ops.cpp` реализует операции над динамическим массивом `int*` — создание, изменение и освобождение, — не обращаясь к `std::cin`; выводом занимается только `array_print`;
- `main.cpp` содержит текстовое меню, чтение и проверку ввода, обработку ошибок и вызов функций `array_ops`.

Указатель и его длина хранятся в `main` как `int* arr` и `std::size_t size`. Операции, изменяющие длину, получают `std::size_t& size` как out-параметр: функции `array_insert` и `array_remove` обновляют длину по ссылке. Функция поиска `array_binary_search` возвращает `bool`, а найденный индекс передаётся через out-параметр `std::size_t& out_index`.

2.2 Состав функций

Таблица 1: Чистые функции модуля `array_ops`

Сигнатура
`int* array_create(std::size_t size)`
`void array_delete(int*& arr)`
`int* array_resize(int* arr, std::size_t size, std::size_t new_size)`
`int* array_insert(int* arr, std::size_t& size, std::size_t pos, int value)`
`int* array_remove(int* arr, std::size_t& size, std::size_t pos)`
`void array_print(const int* arr, std::size_t size)`
`void array_selection_sort(int* arr, std::size_t size)`
`void array_reverse(int* arr, std::size_t size)`
`bool array_binary_search(const int* arr, std::size_t size, int target, std::size_t& out_index)`

2.3 Алгоритм варианта

Пункт 7 выполняет сортировку выбором по возрастанию, затем разворот массива двумя указателями. Сортировка перебирает позиции `i` и находит индекс минимума на отрезке `[i, size)`, меняя `arr[i]` и `arr[minInd]` местами через временную переменную. Разворот сдвигает указатели `left` и `right` к центру, обменивая `arr[left]` и `arr[right]` до их встречи.

2.4 Состав меню

Меню соответствует семейству A, дополнено пунктом 8 для демонстрации поиска:

Таблица 2: Пункты меню

1	Создать массив	5	Изменить размер
2	Напечатать	6	Печать
3	Вставить элемент	7	Алгоритм варианта
4	Удалить элемент	8	Бинарный поиск
		0	Выход

Пункт 8 является осознанным расширением меню семейства A: цель работы прямо требует «алгоритм варианта и поиск с возвратом результата и найденного значения по ссылке», а обязательная функция семейства — это `array_binary_search`. Чтобы поиск выполнялся на упорядоченных данных, пункт 8 создаёт копию массива, сортирует её выбором и выполняет бинарный поиск по копии, не изменяя исходный массив. Копия освобождается сразу после поиска.

2.5 Ввод, ошибки и владение памятью

- Размеры и позиции вводятся во временную переменную типа `long long` и проверяются на отрицательность до приведения к `std::size_t`. Это исключает переполнение при преобразовании отрицательного числа в беззнаковый тип.
- Чтение выполняется через функции `readLongLong` и `readInt`, которые проверяют результат `std::cin` и при ошибке вызывают `clearInput()`: сбрасывают флаги состояния и `std::cin.ignore` до конца строки. Проверка `std::cin.eof()` при выборе пункта меню завершает программу штатно, поэтому перенаправление ввода из файла или пайпа не приводит к зацикливанию.
- Операции, выделяющие память (создание, вставка, удаление, изменение размера, копия для поиска), обёрнуты в `try/catch` по `std::bad_alloc`. При нехватке памяти выводится сообщение, а исходный массив и его длина остаются неизменными.
- Каждая операция выводит результат: сам массив и новую длину. При вставке за конец выводится предупреждение и фактическая позиция вставки. При удалении позиции вне диапазона выводится допустимый диапазон, и массив не изменяется.
- Функции, изменяющие длину, работают по принципу «сначала новая память, затем копирование, затем `delete[]` старой». Поэтому при успешном выделении памяти исключение внутри функции уже не может привести к утечке. `array_delete` обнуляет указатель.

2.6 Сборка и запуск

Проект собирается out-of-source через CMake: библиотека `array_ops`, исполняемый файл `main` и тест `test_variant`. Google Test подключается через `FetchContent` (версия 1.14.0), тесты регистрируются в CTest командой `gtest_discover_tests`.

Листинг 1: Сборка и запуск

```
cmake -S . -B build
cmake --build build
./build/main
ctest --test-dir build --output-on-failure
```

2.7 Исходный код

Система сборки

Листинг 2: CMakeLists.txt

```
# Files
cmake_minimum_required(VERSION 3.16)
project(lab1_variant2 CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

if(MSVC)
    set(LAB_WARNINGS /W4)
else()
    set(LAB_WARNINGS -Wall -Wextra -Wpedantic)
endif()

add_library(array_ops STATIC src/array_ops.cpp)
target_include_directories(array_ops PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/src)
target_compile_options(array_ops PRIVATE ${LAB_WARNINGS})

add_executable(main src/main.cpp)
target_link_libraries(main PRIVATE array_ops)
target_compile_options(main PRIVATE ${LAB_WARNINGS})

# Tests
include(FetchContent)
set(CMP0135 NEW)
FetchContent_Declare(
    googletest
    URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
    DOWNLOAD_EXTRACT_TIMESTAMP TRUE
)
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)

enable_testing()
add_executable(test_variant tests/test_variant.cpp)
target_link_libraries(test_variant PRIVATE array_ops GTest::gtest_main)
```

```
target_compile_options(test_variant PRIVATE ${LAB_WARNINGS})

include(GoogleTest)
gtest_discover_tests(test_variant)
```

Заголовочный файл

Листинг 3: src/array_ops.h

```
#pragma once
#include <cstddef>

int* array_create(std::size_t size);
void array_delete(int*& arr);
int* array_resize(int* arr, std::size_t size, std::size_t new_size);
int* array_insert(int* arr, std::size_t& size, std::size_t pos, int value);
int* array_remove(int* arr, std::size_t& size, std::size_t pos);
void array_print(const int* arr, std::size_t size);

void array_selection_sort(int* arr, std::size_t size);
void array_reverse(int* arr, std::size_t size);
bool array_binary_search(const int* arr, std::size_t size, int target,
    std::size_t& out_index);
```

Реализация операций

Листинг 4: src/array_ops.cpp

```
#include "array_ops.h"
#include <iostream>

int* array_create(std::size_t size) {
    if (size == 0) return nullptr;
    return new int[size]{};
}

void array_delete(int*& arr) {
    delete[] arr;
    arr = nullptr;
}

int* array_resize(int* arr, std::size_t size, std::size_t new_size) {
    int* fresh = (new_size == 0) ? nullptr : new int[new_size]{};
    std::size_t to_copy = (size < new_size) ? size : new_size;
    for (std::size_t i = 0; i < to_copy; ++i)
        fresh[i] = arr[i];
    delete[] arr;
    return fresh;
}

int* array_insert(int* arr, std::size_t& size, std::size_t pos, int value) {
    if (pos > size) pos = size;
    int* fresh = new int[size + 1];
    for (std::size_t i = 0; i < pos; ++i) fresh[i] = arr[i];
    fresh[pos] = value;
    for (std::size_t i = pos; i < size; ++i) fresh[i + 1] = arr[i];
```

```
    delete[] arr;
    ++size;
    return fresh;
}

int* array_remove(int* arr, std::size_t& size, std::size_t pos) {
    if (pos >= size || size == 0) return arr;
    int* fresh = (size - 1 == 0) ? nullptr : new int[size - 1];
    for (std::size_t i = 0, j = 0; i < size; ++i) {
        if (i == pos) continue;
        fresh[j++] = arr[i];
    }
    delete[] arr;
    --size;
    return fresh;
}

void array_print(const int* arr, std::size_t size) {
    if (size == 0 || arr == nullptr) {
        std::cout << "[]\n";
        return;
    }
    std::cout << "[ ";
    for (std::size_t i = 0; i < size; ++i)
        std::cout << arr[i] << (i + 1 < size ? ", " : " ");
    std::cout << "]\n";
}

void array_selection_sort(int* arr, std::size_t size) {
    if (size < 2 || arr == nullptr) {
        return;
    }
    for (std::size_t i = 0; i + 1 < size; ++i) {
        std::size_t minInd = i;
        for (std::size_t j = i + 1; j < size; ++j) {
            if (arr[j] < arr[minInd]) minInd = j;
        }
        if (minInd != i) {
            int tmp = arr[i];
            arr[i] = arr[minInd];
            arr[minInd] = tmp;
        }
    }
}

void array_reverse(int* arr, std::size_t size) {
    if (arr == nullptr || size < 2) return;
    std::size_t left = 0;
    std::size_t right = size - 1;
    while (left < right) {
        int tmp = arr[left];
        arr[left] = arr[right];
        arr[right] = tmp;
```

```
82          ++left;
83          --right;
84      }
85  }
86
87  bool array_binary_search(const int* arr, std::size_t size, int target,
        std::size_t& out_index) {
88      if (arr == nullptr || size == 0) {
89          return false;
90      }
91
92      std::size_t left = 0;
93      std::size_t right = size;
94
95      while (left < right) {
96          std::size_t mid = left + (right - left) / 2;
97          if (arr[mid] == target) {
98              out_index = mid;
99              return true;
100         }
101         if (arr[mid] < target) {
102             left = mid + 1;
103         } else {
104             right = mid;
105         }
106     }
107
108     return false;
109 }
```

Точка входа

Листинг 5: src/main.cpp

```
1   #include "array_ops.h"
2   #include <iostream>
3   #include <limits>
4   #include <new>
5
6   namespace
7   {
8
9       void clearInput()
10      {
11          std::cin.clear();
12          std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
13      }
14
15      bool readLongLong(long long &n)
16      {
17          if (!(std::cin >> n))
18          {
19              clearInput();
20              return false;
21          }
```

```
        return true;
    }

    bool readInt(int &x)
    {
        if (!(std::cin >> x))
        {
            clearInput();
            return false;
        }
        return true;
    }

} // namespace

int main()
{
    int *arr = nullptr;
    std::size_t size = 0;
    bool running = true;

    while (running)
    {
        std::cout << "\n======== Меню ========\n"
                  << "1. Создать массив\n"
                  << "2. Напечатать\n"
                  << "3. Вставить элемент\n"
                  << "4. Удалить элемент\n"
                  << "5. Изменить размер\n"
                  << "6. Печать\n"
                  << "7. Алгоритм варианта сортировка( выбором + разворот)\n"
                  << "8. Бинарный поиск с( предварительной сортировкой выбором
    по копии)\n"
                  << "0. Выход\n"
                  << "Выбор: ";

        int choice;
        if (!(std::cin >> choice))
        {
            if (std::cin.eof())
            {
                std::cout << "\Конецn ввода\n";
                break;
            }
            clearInput();
            std::cout << "Ошибка ввода\n";
            continue;
        }

        switch (choice)
        {
        case 1:
        {
```

```
74              long long rawN;
75              std::cout << "Размер: ";
76              if (!readLongLong(rawN))
77              {
78                  std::cout << "Ошибка\n";
79                  break;
80              }
81              if (rawN < 0)
82              {
83                  std::cout << "Размер не может быть отрицательным\n";
84                  break;
85              }
86              std::size_t n = static_cast<std::size_t>(rawN);
87
88              int *fresh = nullptr;
89              try
90              {
91                  fresh = array_create(n);
92              }
93              catch (const std::bad_alloc &)
94              {
95                  std::cout << "Недостаточно памяти: не удалось выделить массив
    из " << n << " элементов\n";
96                  break;
97              }
98              array_delete(arr);
99              arr = fresh;
100             size = n;
101             std::cout << "Введите " << n << " элементов:\n";
102             for (std::size_t i = 0; i < n; ++i)
103             {
104                 if (!readInt(arr[i]))
105                 {
106                     std::cout << "Ошибка ввода\n";
107                     break;
108                 }
109             }
110             std::cout << "Создан массив размер( " << size << "): ";
111             array_print(arr, size);
112             break;
113         }
114         case 2:
115         case 6:
116             array_print(arr, size);
117             break;
118         case 3:
119         {
120             long long rawPos;
121             int value;
122             std::cout << "Позиция: ";
123             if (!readLongLong(rawPos))
124             {
125                 std::cout << "Ошибка ввода\n";
```

```
126                 break;
127             }
128             if (rawPos < 0)
129             {
130                 std::cout << "Позиция не может быть отрицательной\n";
131                 break;
132             }
133             std::size_t pos = static_cast<std::size_t>(rawPos);
134 
135             std::cout << "Значение: ";
136             if (!readInt(value))
137             {
138                 std::cout << "Ошибка ввода\n";
139                 break;
140             }
141 
142             std::size_t before = size;
143             int *fresh = nullptr;
144             try
145             {
146                 fresh = array_insert(arr, size, pos, value);
147             }
148             catch (const std::bad_alloc &)
149             {
150                 std::cout << "Недостаточно памяти: вставка не выполнена\n";
151                 break;
152             }
153             arr = fresh;
154             std::size_t actual = (pos < before) ? pos : before;
155             if (pos > before)
156             {
157                 std::cout << "Позиция " << pos << " больше длины " << before <<
    " - вставлено в конец\n";
158             }
159             std::cout << "Вставлено " << value << " на позицию " << actual <<
    ", новый размер " << size << ": ";
160             array_print(arr, size);
161             break;
162         }
163         case 4:
164         {
165             long long rawPos;
166             std::cout << "Позиция: ";
167             if (!readLongLong(rawPos))
168             {
169                 std::cout << "Ошибка\n";
170                 break;
171             }
172             if (rawPos < 0)
173             {
174                 std::cout << "Позиция не может быть отрицательной\n";
175                 break;
176             }
```

```
177             std::size_t pos = static_cast<std::size_t>(rawPos);
178
179             if (size == 0)
180             {
181                 std::cout << "Массив пуст, удалять нечего\n";
182                 break;
183             }
184             if (pos >= size)
185             {
186                 std::cout << "Позиция " << pos << " вне диапазона [0, " << size
    - 1 << "], массив не изменён\n";
187                 break;
188             }
189
190             int removed = arr[pos];
191             int *fresh = nullptr;
192             try
193             {
194                 fresh = array_remove(arr, size, pos);
195             }
196             catch (const std::bad_alloc &)
197             {
198                 std::cout << "Недостаточно памяти: удаление не выполнено\n";
199                 break;
200             }
201             arr = fresh;
202             std::cout << "Удалён элемент " << removed << " с позиции " << pos
    << ", новый размер " << size << ": ";
203             array_print(arr, size);
204             break;
205         }
206         case 5:
207         {
208             long long rawNew;
209             std::cout << "Новый размер: ";
210             if (!readLongLong(rawNew))
211             {
212                 std::cout << "Ошибка\n";
213                 break;
214             }
215             if (rawNew < 0)
216             {
217                 std::cout << "Размер не может быть отрицательным\n";
218                 break;
219             }
220             std::size_t newSize = static_cast<std::size_t>(rawNew);
221
222             int *fresh = nullptr;
223             try
224             {
225                 fresh = array_resize(arr, size, newSize);
226             }
227             catch (const std::bad_alloc &)
```

```
228             {
229                 std::cout << "Недостаточно памяти: массив не изменён\n";
230                 break;
231             }
232             arr = fresh;
233             size = newSize;
234             std::cout << "После изменения размера размер( " << size << "): ";
235             array_print(arr, size);
236             break;
237         }
238         case 7:
239         {
240             array_selection_sort(arr, size);
241             std::cout << "После сортировки выбором: ";
242             array_print(arr, size);
243 
244             array_reverse(arr, size);
245             std::cout << "После разворота: ";
246             array_print(arr, size);
247             break;
248         }
249         case 8:
250         {
251             int target;
252             std::cout << "Число для поиска: ";
253             if (!readInt(target))
254             {
255                 break;
256             }
257 
258             int *copy = nullptr;
259             try
260             {
261                 copy = array_create(size);
262             }
263             catch (const std::bad_alloc &)
264             {
265                 std::cout << "Недостаточно памяти: поиск не выполнен\n";
266                 break;
267             }
268             for (std::size_t i = 0; i < size; ++i)
269             {
270                 copy[i] = arr[i];
271             }
272             array_selection_sort(copy, size);
273             std::cout << "Отсортированный массив: ";
274             array_print(copy, size);
275 
276             std::size_t index = 0;
277             if (array_binary_search(copy, size, target, index))
278             {
279                 std::cout << "Найдено на позиции " << index << '\n';
280             }
```

```
            else
            {
                std::cout << "Не найдено\n";
            }
            array_delete(copy);
            break;
        }
        case 0:
            std::cout << "Выход\n";
            running = false;
            break;
        default:
            std::cout << "Нет такого пункта\n";
        }
    }

    array_delete(arr);
    return 0;
}
```

Тесты

Листинг 6: tests/test_variant.cpp

```
#include <gtest/gtest.h>
#include "array_ops.h"

TEST(ArrayOps, CreateAndDelete)
{
    int *a = array_create(5);
    ASSERT_NE(a, nullptr);
    for (std::size_t i = 0; i < 5; ++i)
        EXPECT_EQ(a[i], 0);
    array_delete(a);
    EXPECT_EQ(a, nullptr);
}

TEST(ArrayOps, CreateZero)
{
    int *a = array_create(0);
    EXPECT_EQ(a, nullptr);
    array_delete(a);
    EXPECT_EQ(a, nullptr);
}

TEST(ArrayOps, InsertMiddle)
{
    std::size_t size = 3;
    int *a = array_create(3);
    a[0] = 1;
    a[1] = 2;
    a[2] = 3;
    a = array_insert(a, size, 1, 99);
    ASSERT_EQ(size, 4u);
    EXPECT_EQ(a[0], 1);
```

```
32      EXPECT_EQ(a[1], 99);
33      EXPECT_EQ(a[2], 2);
34      EXPECT_EQ(a[3], 3);
35      array_delete(a);
36  }
37
38  TEST(ArrayOps, InsertEnd)
39  {
40      std::size_t size = 2;
41      int *a = array_create(2);
42      a[0] = 7;
43      a[1] = 8;
44      a = array_insert(a, size, 2, 9);
45      ASSERT_EQ(size, 3u);
46      EXPECT_EQ(a[2], 9);
47      array_delete(a);
48  }
49
50  TEST(ArrayOps, RemoveMiddle)
51  {
52      std::size_t size = 4;
53      int *a = array_create(4);
54      a[0] = 1;
55      a[1] = 2;
56      a[2] = 3;
57      a[3] = 4;
58      a = array_remove(a, size, 1);
59      ASSERT_EQ(size, 3u);
60      EXPECT_EQ(a[0], 1);
61      EXPECT_EQ(a[1], 3);
62      EXPECT_EQ(a[2], 4);
63      array_delete(a);
64  }
65
66  TEST(ArrayOps, RemoveLast)
67  {
68      std::size_t size = 3;
69      int *a = array_create(3);
70      a[0] = 1;
71      a[1] = 2;
72      a[2] = 3;
73      a = array_remove(a, size, 2);
74      ASSERT_EQ(size, 2u);
75      EXPECT_EQ(a[0], 1);
76      EXPECT_EQ(a[1], 2);
77      array_delete(a);
78  }
79
80  TEST(ArrayOps, InsertFront)
81  {
82      std::size_t size = 2;
83      int *a = array_create(2);
84      a[0] = 1;
```

```
    a[1] = 2;
    a = array_insert(a, size, 0, 99);
    ASSERT_EQ(size, 3u);
    EXPECT_EQ(a[0], 99);
    EXPECT_EQ(a[1], 1);
    EXPECT_EQ(a[2], 2);
    array_delete(a);
}

TEST(ArrayOps, InsertBeyondEndAppends)
{
    std::size_t size = 2;
    int *a = array_create(2);
    a[0] = 1;
    a[1] = 2;
    a = array_insert(a, size, 100, 9);
    ASSERT_EQ(size, 3u);
    EXPECT_EQ(a[2], 9);
    array_delete(a);
}

TEST(ArrayOps, InsertIntoEmpty)
{
    std::size_t size = 0;
    int *a = array_create(0);
    a = array_insert(a, size, 0, 42);
    ASSERT_EQ(size, 1u);
    ASSERT_NE(a, nullptr);
    EXPECT_EQ(a[0], 42);
    array_delete(a);
}

TEST(ArrayOps, RemoveOutOfRangeKeepsArray)
{
    std::size_t size = 3;
    int *a = array_create(3);
    a[0] = 1;
    a[1] = 2;
    a[2] = 3;
    int *same = array_remove(a, size, 7);
    EXPECT_EQ(same, a);
    EXPECT_EQ(size, 3u);
    array_delete(a);
}

TEST(ArrayOps, ResizeGrowZeroFills)
{
    int *a = array_create(2);
    a[0] = 5;
    a[1] = 6;
    a = array_resize(a, 2, 5);
    ASSERT_NE(a, nullptr);
    EXPECT_EQ(a[0], 5);
```

```
    EXPECT_EQ(a[1], 6);
    EXPECT_EQ(a[2], 0);
    EXPECT_EQ(a[3], 0);
    EXPECT_EQ(a[4], 0);
    array_delete(a);
}

TEST(ArrayOps, ResizeShrinkTruncates)
{
    int *a = array_create(4);
    for (std::size_t i = 0; i < 4; ++i)
        a[i] = static_cast<int>(i) + 1;
    a = array_resize(a, 4, 2);
    ASSERT_NE(a, nullptr);
    EXPECT_EQ(a[0], 1);
    EXPECT_EQ(a[1], 2);
    array_delete(a);
}

TEST(ArrayOps, ResizeToZeroReturnsNull)
{
    int *a = array_create(3);
    a[0] = 1;
    a[1] = 2;
    a[2] = 3;
    a = array_resize(a, 3, 0);
    EXPECT_EQ(a, nullptr);
}

TEST(ArrayOps, ResizeFromNull)
{
    int *a = array_resize(nullptr, 0, 3);
    ASSERT_NE(a, nullptr);
    EXPECT_EQ(a[0], 0);
    EXPECT_EQ(a[1], 0);
    EXPECT_EQ(a[2], 0);
    array_delete(a);
}

TEST(Variant2, SelectionSort)
{
    int a[] = {3, 1, 4, 1, 5};
    array_selection_sort(a, 5);
    int expected[] = {1, 1, 3, 4, 5};
    for (int i = 0; i < 5; ++i)
        EXPECT_EQ(a[i], expected[i]);
}

TEST(Variant2, SortEdgeCases)
{
    array_selection_sort(nullptr, 0);
    int one[] = {42};
    array_selection_sort(one, 1);
```

```
191     EXPECT_EQ(one[0], 42);
192 }
193
194 TEST(Variant2, Reverse)
195 {
196     int a[] = {1, 2, 3, 4, 5};
197     array_reverse(a, 5);
198     int expected[] = {5, 4, 3, 2, 1};
199     for (int i = 0; i < 5; ++i)
200         EXPECT_EQ(a[i], expected[i]);
201 }
202
203 TEST(Variant2, ReverseEvenAndOne)
204 {
205     int a[] = {1, 2, 3, 4};
206     array_reverse(a, 4);
207     EXPECT_EQ(a[0], 4);
208     EXPECT_EQ(a[1], 3);
209     EXPECT_EQ(a[2], 2);
210     EXPECT_EQ(a[3], 1);
211
212     int one[] = {7};
213     array_reverse(one, 1);
214     EXPECT_EQ(one[0], 7);
215 }
216
217 TEST(Variant2, FullAlgorithm)
218 {
219     int a[] = {3, 1, 4, 1, 5};
220     array_selection_sort(a, 5);
221     array_reverse(a, 5);
222     int expected[] = {5, 4, 3, 1, 1};
223     for (int i = 0; i < 5; ++i)
224         EXPECT_EQ(a[i], expected[i]);
225 }
226
227 TEST(Variant2, EmptyAlgorithm)
228 {
229     array_selection_sort(nullptr, 0);
230     array_reverse(nullptr, 0);
231     SUCCEED();
232 }
233
234 TEST(BinarySearch, FoundMiddle)
235 {
236     int a[] = {1, 3, 5, 7, 9};
237     std::size_t idx = 0;
238     ASSERT_TRUE(array_binary_search(a, 5, 5, idx));
239     EXPECT_EQ(idx, 2u);
240 }
241
242 TEST(BinarySearch, FoundLast)
243 {
```

```
244     int a[] = {1, 3, 5, 7, 9};
245     std::size_t idx = 0;
246     ASSERT_TRUE(array_binary_search(a, 5, 9, idx));
247     EXPECT_EQ(idx, 4u);
248 }
249
250 TEST(BinarySearch, DuplicatesFindAnOccurrence)
251 {
252     int a[] = {1, 1, 3, 4, 5};
253     std::size_t idx = 0;
254     ASSERT_TRUE(array_binary_search(a, 5, 1, idx));
255     EXPECT_EQ(a[idx], 1);
256 }
257
258 TEST(BinarySearch, NotFoundBelowAndAbove)
259 {
260     int a[] = {1, 3, 5, 7, 9};
261     std::size_t idx = 0;
262     EXPECT_FALSE(array_binary_search(a, 5, 0, idx));
263     EXPECT_FALSE(array_binary_search(a, 5, 10, idx));
264 }
265
266 TEST(BinarySearch, NotFoundBetween)
267 {
268     int a[] = {1, 3, 5, 7, 9};
269     std::size_t idx = 0;
270     EXPECT_FALSE(array_binary_search(a, 5, 4, idx));
271 }
272
273 TEST(BinarySearch, EmptyArray)
274 {
275     std::size_t idx = 0;
276     EXPECT_FALSE(array_binary_search(nullptr, 0, 42, idx));
277 }
```

3 Тестирование

Тесты реализованы на Google Test в файле `tests/test_variant.cpp` и подключены к CTest через `gtest_discover_tests`. Всего написано **26 тестов** при требовании не менее 10, распределённых по трём наборам.

Таблица 3: Наборы тестов

Набор	Тестов	Что проверяет
`ArrayOps`	14	Создание, освобождение, вставка, удаление, изменение размера
`Variant2`	6	Сортировка выбором, разворот, полный алгоритм варианта
`BinarySearch`	6	Бинарный поиск, в том числе на дубликатах и пустом массиве

3.1 Покрытые сценарии

- **Память:** создание массива с нулевой инициализацией, создание размера 0 с возвратом `nullptr`, обнуление указателя после `array_delete`.
- **Вставка:** в начало, в середину, в конец, за пределами конца (с фактической позицией `size`), в пустой массив.
- **Удаление:** из середины, последнего элемента, позиции вне диапазона — массив и длина не изменяются.
- **Изменение размера:** рост с заполнением новых элементов нулями, усечение, переход к размеру 0 с возвратом `nullptr`, изменение размера из `nullptr`.
- **Алгоритм варианта:** сортировка выбором, сортировка массива длины 1 и `nullptr`, разворот массива чётной длины и длины 1, полный алгоритм на контрольном примере, обработка пустого массива.
- **Поиск:** найденное значение в середине и в конце, повторяющиеся элементы (проверяется факт нахождения одного из вхождений), отсутствие значения ниже, выше и между границами диапазона, пустой массив.

3.2 Контрольный пример

Алгоритм варианта проверен на примере из задания. Сортировка выбором по возрастанию, затем разворот:

Исходный массив	После сортировки	После разворота
`{3, 1, 4, 1, 5}`	`{1, 1, 3, 4, 5}`	`{5, 4, 3, 1, 1}`

3.3 Результат

Все 26 тестов из трёх наборов проходят успешно. Сборка выполняется без предупреждений компилятора при включённых флагах `-Wall -Wextra -Wpedantic` (GCC) и `/W4` (MSVC).

4 Выводы

В ходе работы решены задачи, поставленные в лабораторной работе №1:

1. Написана консольная программа семейства A для варианта 2: интерактивный справочник, работающий с линейным динамическим массивом через текстовое меню.
2. Вся логика вынесена в библиотеку чистых функций `array_ops`, использующую только `new[]` / `delete[]`. Запрещённые конструкции — классы, конструкторы, семантика копирования и перемещения, `std::string`, `std::vector`, `std::array`, умные указатели — не используются.
3. Реализованы сортировка выбором по возрастанию и разворот массива двумя указателями; на контрольном примере задания получен ожидаемый результат `{3, 1, 4, 1, 5}` → `{1, 1, 3, 4, 5}` → `{5, 4, 3, 1, 1}`.

4. Обеспечены требования к корректности: отсутствие утечек памяти за счёт схемы «новая память, копирование, освобождение старой» и обнуления указателя, проверка `std::cin` с восстановлением состояния потока, штатная обработка конца ввода, корректная работа с массивом длины 0 и перехват `std::bad_alloc` с сохранением исходных данных.

5. Собран проект CMake (C++20) и написано 26 тестов Google Test в трёх наборах, подключённых к CTest, — при требовании не менее 10 тестов.

Практический результат: получен рабочий, собираемый и полностью покрытый тестами пример ручного управления динамической памятью через указатели, ссылки и out-параметры.

5 Список литературы

1. ISO/IEC 14882:2020. Programming languages — C++. Geneva: ISO, 2020. Разделы 6.7.5 (initializer lists), 11.2 (array-to-pointer decay), 17.4 (allocators).

2. Б. Страуструп. Язык программирования C++. Спецификация ISO/IEC 14882:2020. — М.: ДМК Пресс.

3. Документация Google Test: работа с массивами, выделенными через `new[]` / `delete[]`, регистрация тестов в CTest командой `gtest_discover_tests`.

4. Справочник по C++: `std::bad_alloc`, `std::cin::clear`, `std::cin.ignore`, `std::size_t`.